

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – DESIGN AND CONNECTIVITY

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 1 – DESIGN AND CONNECTIVITY
Weightage:		Total Marks:	40 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	Dharshini H	Reg. No.:	192324268
Section / Batch:	B.Tech AI and DS	Date of Submission:	04-08-2026
Faculty In-charge:	Dr. B.SHYAMALA GOWRI	Project Mode:	Individual Submission

Code of Conduct

I Dharshini H 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

Requirement Analysis (5 Marks)	ER Diagram Design (10 Marks)	Table Design & Normalization (5 Marks)	Database Connectivity using Python (20 Marks)	CRUD Operations (5 Marks)	Total (35 Marks)

Scenario 1: University Student Management System

A university manages information related to students, faculty members, departments, courses, and examinations. Currently, student information is stored in spreadsheets by different departments, resulting in duplicated records, inconsistent student IDs, and difficulties in generating reports. The university administration wants to develop a centralized database system that can store student details, course enrollments, faculty assignments, attendance records, and examination results. Students should be able to register for multiple courses, while each course may have multiple students enrolled. Faculty members may teach several courses within a semester. The management also requires reports such as department-wise student strength, faculty workload, and student performance statistics. Design an appropriate database schema by identifying entities, attributes, primary keys, foreign keys, and relationships. Connect the database using Python and demonstrate CRUD operations for managing student records.

University Student Management System

1. Entities and Attributes

1. Student

Attribute	Data Type	Description
Student ID (PK)	INT	Unique student ID
First_Name	VARCHAR(50)	Student first name
Last_Name	VARCHAR(50)	Student last name
Gender	VARCHAR(10)	Gender
DOB	DATE	Date of Birth
Email	VARCHAR(100)	Email address
Phone	VARCHAR(15)	Contact number
Department_ID (FK)	INT	Department reference

2. Department

Attribute	Data Type	Description
Department_ID (PK)	INT	Department ID
Department_Name	VARCHAR(100)	Department Name
HOD	VARCHAR(100)	Head of Department

3. Faculty

Attribute	Data Type	Description
Faculty_ID (PK)	INT	Faculty ID
Faculty_Name	VARCHAR(100)	Faculty Name
Email	VARCHAR(100)	Faculty Email
Phone	VARCHAR(15)	Phone Number
Department_ID (FK)	INT	Department

4. Course

Attribute	Data Type	Description
Course_ID (PK)	INT	Course ID
Course_Name	VARCHAR(100)	Course Name
Credits	INT	Course Credits
Department_ID (FK)	INT	Department

5. Enrollment

Attribute	Data Type	Description
Enrollment_ID (PK)	INT	Enrollment ID
Student_ID (FK)	INT	Student
Course_ID (FK)	INT	Course
Semester	VARCHAR(20)	Semester

6. Faculty_Assignment

Attribute	Data Type	Description
Assignment_ID (PK)	INT	Assignment ID
Faculty_ID (FK)	INT	Faculty
Course_ID (FK)	INT	Course
Semester	VARCHAR(20)	Semester

7. Attendance

Attribute	Data Type	Description
Attendance_ID (PK)	INT	Attendance ID
Student_ID (FK)	INT	Student
Course_ID (FK)	INT	Course
Attendance_Percentage	DECIMAL(5,2)	Attendance Percentage

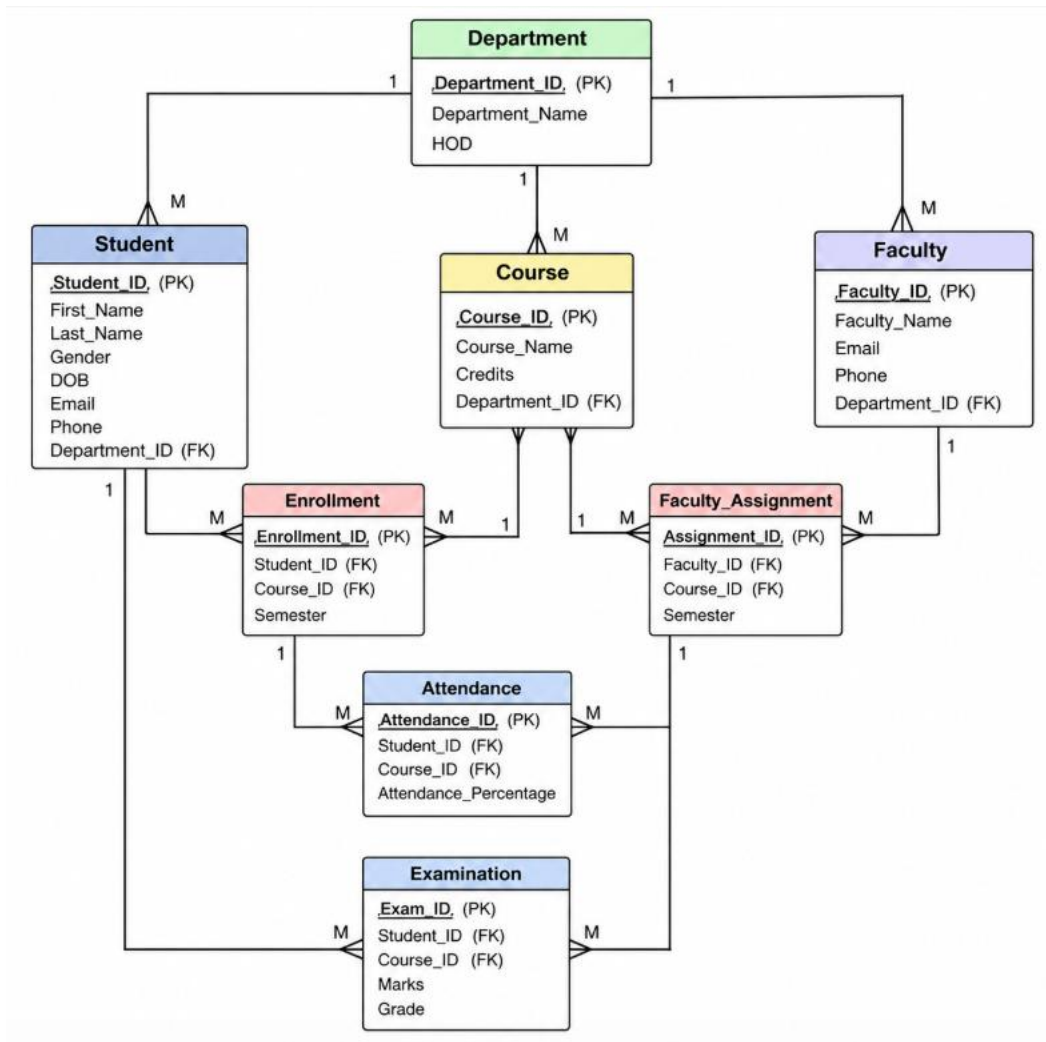
8. Examination

Attribute	Data Type	Description
Exam_ID (PK)	INT	Exam ID
Student_ID (FK)	INT	Student
Course_ID (FK)	INT	Course
Marks	INT	Marks Obtained
Grade	CHAR(2)	Grade

2. Relationships

Relationship	Type
Department → Student	One-to-Many
Department → Faculty	One-to-Many
Department → Course	One-to-Many
Student ↔ Course	Many-to-Many (through Enrollment)
Faculty ↔ Course	Many-to-Many (through Faculty_Assignment)
Student → Attendance	One-to-Many
Student → Examination	One-to-Many
Course → Attendance	One-to-Many
Course → Examination	One-to-Many

3. ER Diagram



4. CRUD Operation

Step 1: Python Database Connectivity

```
CO2 > AT1_1.py > ...
1 import sqlite3
2 conn = sqlite3.connect("university.db")
3 cur = conn.cursor()
4 print("Database Connected Successfully")
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\dhars\Documents\Query processing\lab_questions> & C:\Users\dhars\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dhars/Documents/Query processing/lab_questions/CO2/AT1_1.py"
Database Connected Successfully
PS C:\Users\dhars\Documents\Query processing\lab_questions> █
```

Step 2: Create Table Using Python

```
CO2 > AT1_1.py > ...
1 import sqlite3
2 conn = sqlite3.connect("university.db")
3 cur = conn.cursor()
4 cur.execute("""
5 CREATE TABLE IF NOT EXISTS Student(
6     Student_ID INTEGER PRIMARY KEY,
7     First_Name TEXT,
8     Last_Name TEXT,
9     Gender TEXT,
10    DOB TEXT,
11    Email TEXT,
12    Phone TEXT,
13    Department_ID INTEGER
14 )
15 """)
16 conn.commit()
17 print("Student Table Created Successfully")
18 conn.close()
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\dhars\Documents\Query processing\lab_questions> & C:\Users\dhars\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dhars/Documents/Query processing/lab_questions/CO2/AT1_1.py"
Student Table Created Successfully
PS C:\Users\dhars\Documents\Query processing\lab_questions> █
```

Step 3: Insert Records (Create)

```
students = [  
    (101, 'Rahul', 'Sharma', 'Male', '2003-05-20', 'rahul@gmail.com', '9876543210', 1),  
    (102, 'Priya', 'Kumar', 'Female', '2002-09-15', 'priya@gmail.com', '9876543211', 2),  
    (103, 'Arun', 'Raj', 'Male', '2003-01-10', 'arun@gmail.com', '9876543212', 1)  
]
```

Step 4: Read Records

```
print("\n----- Student Records -----")  
  
cur.execute("SELECT * FROM Student")  
  
rows = cur.fetchall()  
  
for row in rows:  
    print(row)
```

```
----- Student Records -----  
(101, 'Rahul', 'Sharma', 'Male', '2003-05-20', 'rahul@gmail.com', '9876543210', 1)  
(102, 'Priya', 'Kumar', 'Female', '2002-09-15', 'priya@gmail.com', '9876543211', 2)  
(103, 'Arun', 'Raj', 'Male', '2003-01-10', 'arun@gmail.com', '9876543212', 1)
```

Step 5: Update Record

```
cur.execute("""  
UPDATE Student  
SET Email='rahul123@gmail.com'  
WHERE Student_ID=101  
""")  
conn.commit()  
print("\nRecord Updated Successfully")  
print("\n----- Records After Update -----")  
cur.execute("SELECT * FROM Student")  
rows = cur.fetchall()  
for row in rows:  
    print(row)
```

Record Updated Successfully

```
----- Records After Update -----
(101, 'Rahul', 'Sharma', 'Male', '2003-05-20', 'rahul123@gmail.com', '9876543210', 1)
(102, 'Priya', 'Kumar', 'Female', '2002-09-15', 'priya@gmail.com', '9876543211', 2)
(103, 'Arun', 'Raj', 'Male', '2003-01-10', 'arun@gmail.com', '9876543212', 1)
```

Step 6: Delete Record

```
cur.execute("""
DELETE FROM Student
WHERE Student_ID=101
""")
conn.commit()
print("\nRecord Deleted Successfully")
print("\n----- Records After Delete -----")
cur.execute("SELECT * FROM Student")
rows = cur.fetchall()
for row in rows:
    print(row)
```

Record Deleted Successfully

```
----- Records After Delete -----
(102, 'Priya', 'Kumar', 'Female', '2002-09-15', 'priya@gmail.com', '9876543211', 2)
(103, 'Arun', 'Raj', 'Male', '2003-01-10', 'arun@gmail.com', '9876543212', 1)
```

Step 7: Close Connection

```
conn.close()

print("\nDatabase Connection Closed")
```


Scenario 6: Banking and Customer Account Management System

A commercial bank manages savings accounts, current accounts, loans, and customer transactions. Customers may own multiple accounts, and each account contains transaction records such as deposits, withdrawals, and transfers. The bank requires a secure database system to store customer information, account details, branch information, and loan records. Managers need reports on customer balances, transaction histories, and loan repayments. The system must support quick retrieval of information and ensure data consistency. Design the database schema, define relationships between customers and accounts, and implement Python-based database connectivity for account management operations.

1. Entities and Attributes

Customer

Attribute	Key
Customer_ID	Primary Key
Customer_Name	
Gender	
Phone	
Email	
Address	

Account

Attribute	Key
Account_No	Primary Key
Account_Type	
Balance	
Customer_ID	Foreign Key
Branch_ID	Foreign Key

Branch

Attribute	Key
Branch_ID	Primary Key
Branch_Name	
Branch_Address	
IFSC_Code	

Transaction

Attribute	Key
Transaction_ID	Primary Key
Account_No	Foreign Key
Transaction_Type	
Amount	
Transaction_Date	

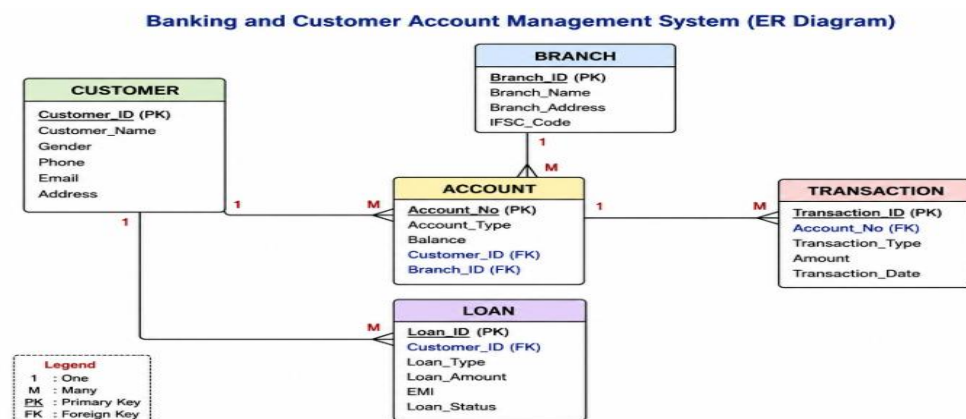
Loan

Attribute	Key
Loan_ID	Primary Key
Customer_ID	Foreign Key
Loan_Type	
Loan_Amount	
EMI	
Loan_Status	

2. Relationships

- One Customer → Many Accounts (1:M)
- One Branch → Many Accounts (1:M)
- One Account → Many Transactions (1:M)
- One Customer → Many Loans (1:M)

3. ER Diagram



4. Python Database Connectivity (SQLite)

CO2 >  AT2_database.py > ...

```
1 import sqlite3
2 conn = sqlite3.connect("bank.db")
3 cur = conn.cursor()
4 print("Database Connected Successfully")
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\dhars\Documents\Query processing\lab_questions> & C:\Users\dhars\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dhars/Documents/Query processing/lab_questions/CO2/AT2_database.py"
Database Connected Successfully
PS C:\Users\dhars\Documents\Query processing\lab_questions> █
```

5. Python Program (Create Table + CRUD Operations)

```
# -----
# Create Customer Table
# -----
cur.execute("""
CREATE TABLE IF NOT EXISTS Customer(
    Customer_ID INTEGER PRIMARY KEY,
    Customer_Name TEXT,
    Gender TEXT,
    Phone TEXT,
    Email TEXT,
    Address TEXT
)
""")
print("Customer Table Created Successfully")
```

```

# -----
# Insert Records (CREATE)
# -----
customers = [
    (101, "Rahul", "Male", "9876543210", "rahul@gmail.com", "Chennai"),
    (102, "Priya", "Female", "9876543211", "priya@gmail.com", "Coimbatore"),
    (103, "Arun", "Male", "9876543212", "arun@gmail.com", "Madurai")
]
cur.execute("DELETE FROM Customer")
cur.executemany("""
INSERT INTO Customer
VALUES(?,?,?,?,,?)
""", customers)
conn.commit()
print("\nCustomer Records Inserted Successfully")

```

```

# -----
# Read Records
# -----
print("\n----- Customer Details -----")
cur.execute("SELECT * FROM Customer")
rows = cur.fetchall()
for row in rows:
    print(row)

```

Customer Records Inserted Successfully

```

----- Customer Details -----
(101, 'Rahul', 'Male', '9876543210', 'rahul@gmail.com', 'Chennai')
(102, 'Priya', 'Female', '9876543211', 'priya@gmail.com', 'Coimbatore')
(103, 'Arun', 'Male', '9876543212', 'arun@gmail.com', 'Madurai')

```

```

# -----
# Update Record
# -----
cur.execute("""
UPDATE Customer
SET Phone='9999999999'
WHERE Customer_ID=101
""")
conn.commit()
print("\nCustomer Record Updated Successfully")
print("\n----- Updated Customer Details -----")
cur.execute("SELECT * FROM Customer")
rows = cur.fetchall()
for row in rows:
    print(row)

```

Customer Record Updated Successfully

```
----- Updated Customer Details -----  
(101, 'Rahul', 'Male', '9999999999', 'rahul@gmail.com', 'Chennai')  
(102, 'Priya', 'Female', '9876543211', 'priya@gmail.com', 'Coimbatore')  
(103, 'Arun', 'Male', '9876543212', 'arun@gmail.com', 'Madurai')
```

```
# -----  
# Delete Record  
# -----  
cur.execute("""  
DELETE FROM Customer  
WHERE Customer_ID=101  
""")  
conn.commit()  
print("\nCustomer Record Deleted Successfully")  
print("\n----- Customer Records After Delete -----")  
cur.execute("SELECT * FROM Customer")  
rows = cur.fetchall()  
for row in rows:  
    print(row)  
# -----  
# Close Connection  
# -----  
conn.close()  
print("\nDatabase Connection Closed")
```